

Programowanie_Objektowe

1.0.0

Generated by Doxygen 1.14.0

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Konfiguracja	??
ObiektSISO	??
ModelARX	??
ObiektStatyczny	??
PetlaUAR	??
RegulatorPID	??
Sygnal	??
Dekorator	??
DekoratorNasycenie	??
DekoratorProstokat	??
DekoratorSinus	??
DekoratorSzumBialy	??
DekoratorTrojkat	??
Nasycenie	??
Prostokat	??
Sinus	??
Stala	??
SygnalStaly	??
SygnalZlozony	??
SzumBialy	??
Trojkat	??

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Dekorator	Abstract base decorator for signal processing	??
DekoratorNasycenie	Saturation decorator limits signal amplitude to a maximum value	??
DekoratorProstokat	Rectangular wave signal decorator	??
DekoratorSinus	Sinusoidal wave signal decorator	??
DekoratorSzumBialy	White noise signal decorator	??
DekoratorTrojkat	Triangular wave signal decorator	??
Konfiguracja	Holds configuration settings for the control system simulation	??
ModelARX	ARX (AutoRegressive with eXogenous input) model implementation	??
Nasycenie		??
ObiektSISO	Abstract SISO system	??
ObiektStatyczny	Represents a static SISO object with output range limits	??
PetlaUAR	Represents a control loop composed of multiple SISO components	??
Prostokat		??
RegulatorPID	PID controller with configurable modes (P/PI/PID)	??
Sinus		??
Stala		??
Sygnal	Interface for signal generators	??
SygnalStaly	Constant value signal	??
SygnalZlozony		??
SzumBialy		??
Trojkat		??

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

Programowanie_Objektowe_04/ Dekorator.h	??
Base decorator class for signal processing (Decorator Pattern)	??
Programowanie_Objektowe_04/ DekoratorNasycenie.h	??
Decorator that applies saturation (clipping) to a signal	??
Programowanie_Objektowe_04/ DekoratorProstokat.h	??
Decorator that applies rectangular wave modulation	??
Programowanie_Objektowe_04/ DekoratorSinus.h	??
Decorator that applies sinusoidal modulation	??
Programowanie_Objektowe_04/ DekoratorSzumBialy.h	??
Decorator that adds white noise to a signal	??
Programowanie_Objektowe_04/ DekoratorTrojkat.h	??
Decorator that applies triangular wave modulation	??
Programowanie_Objektowe_04/ Deserializacja.h	??
Programowanie_Objektowe_04/ Konfiguracja.h	??
Configuration class for the control loop simulation	??
Programowanie_Objektowe_04/ ModelARX.h	??
Programowanie_Objektowe_04/ Nasycenie.h	??
Programowanie_Objektowe_04/ ObiektSISO.h	??
SISO (Single Input Single Output) system interface	??
Programowanie_Objektowe_04/ ObiektStatyczny.h	??
Static object model for control system simulation	??
Programowanie_Objektowe_04/ PetlaUAR.h	??
Control loop for automatic regulation system (UAR)	??
Programowanie_Objektowe_04/ Prostokat.h	??
Programowanie_Objektowe_04/ RegulatorPID.h	??
PID controller implementation	??
Programowanie_Objektowe_04/ Sinus.h	??
Programowanie_Objektowe_04/ Stala.h	??
Programowanie_Objektowe_04/ Sygnal.h	??
Abstract base class for all signal types	??
Programowanie_Objektowe_04/ SygnalStaly.h	??
Constant (DC) signal implementation	??
Programowanie_Objektowe_04/ SygnalZlozony.h	??
Programowanie_Objektowe_04/ SzumBialy.h	??
Programowanie_Objektowe_04/ TestyKonfiguracji.h	??
Declaration of test functions for the control system components	??
Programowanie_Objektowe_04/ Trojkat.h	??

Chapter 4

Class Documentation

4.1 Dekorator Class Reference

Abstract base decorator for signal processing.

```
#include <Dekorator.h>
```

Inheritance diagram for Dekorator:

Collaboration diagram for Dekorator:

Public Member Functions

- [Dekorator](#) (std::unique_ptr< [Signal](#) > k)
Construct a new [Dekorator](#) object.
- double [symuluj](#) (int t) override=0
Simulate signal value at time t.
- void [serializuj](#) (std::ostream &out) const override=0
Serialize signal to output stream.
- std::unique_ptr< [Signal](#) > [klonuj](#) () const override=0
Clone the signal object.

Protected Attributes

- std::unique_ptr< [Signal](#) > [komponent](#)

4.1.1 Detailed Description

Abstract base decorator for signal processing.

Implements the Decorator Pattern to dynamically add behavior to [Signal](#) objects

4.1.2 Constructor & Destructor Documentation

4.1.2.1 Dekorator()

```
Dekorator::Dekorator (  
    std::unique_ptr< Signal > k)
```

Construct a new [Dekorator](#) object.

Parameters

<u><code>k</code></u>	Unique pointer to the Signal to be decorated
-----------------------	--

4.1.3 Member Function Documentation**4.1.3.1 klonuj()**

```
std::unique_ptr< Signal > Dekorator::klonuj () const [override], [pure virtual]
```

Clone the signal object.

Returns

std::unique_ptr<Signal> Copy of the signal

Implements [Signal](#).

Implemented in [DekoratorNasycenie](#), [DekoratorProstokat](#), [DekoratorSinus](#), [DekoratorSzumBialy](#), and [DekoratorTrojkat](#).

4.1.3.2 serializuj()

```
void Dekorator::serializuj (
    std::ostream & out) const [override], [pure virtual]
```

Serialize signal to output stream.

Parameters

<u><code>out</code></u>	Output stream
-------------------------	---------------

Implements [Signal](#).

Implemented in [DekoratorNasycenie](#), [DekoratorProstokat](#), [DekoratorSinus](#), [DekoratorSzumBialy](#), and [DekoratorTrojkat](#).

4.1.3.3 symuluj()

```
double Dekorator::symuluj (
    int t) [override], [pure virtual]
```

Simulate signal value at time t.

Parameters

<u><code>t</code></u>	Time step
-----------------------	-----------

Returns

double Signal value

Implements [Signal](#).

Implemented in [DekoratorNasycenie](#), [DekoratorProstokat](#), [DekoratorSinus](#), [DekoratorSzumBialy](#), and [DekoratorTrojkat](#).

4.1.4 Member Data Documentation

4.1.4.1 komponent

`std::unique_ptr<Sygnal> Dekorator::komponent [protected]`

Wrapped signal component being decorated

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/[Dekorator.h](#)
- Programowanie_Objektowe_04/[Dekorator.cpp](#)

4.2 DekoratorNasycenie Class Reference

Saturation decorator limits signal amplitude to a maximum value.

`#include <DekoratorNasycenie.h>`

Inheritance diagram for DekoratorNasycenie:

Collaboration diagram for DekoratorNasycenie:

Public Member Functions

- [DekoratorNasycenie](#) (std::unique_ptr< [Sygnal](#) > k, double max)
Construct a new saturation decorator.
- double [symuluj](#) (int t) override
Simulate signal value at time t.
- void [serializuj](#) (std::ostream &out) const override
Serialize signal to output stream.
- std::unique_ptr< [Sygnal](#) > [klonuj](#) () const override
Clone the signal object.

Public Member Functions inherited from [Dekorator](#)

- [Dekorator](#) (std::unique_ptr< [Sygnal](#) > k)
Construct a new [Dekorator](#) object.

Additional Inherited Members

Protected Attributes inherited from [Dekorator](#)

- std::unique_ptr< [Sygnal](#) > [komponent](#)

4.2.1 Detailed Description

Saturation decorator limits signal amplitude to a maximum value.

4.2.2 Constructor & Destructor Documentation

4.2.2.1 DekoratorNasycenie()

```
DekoratorNasycenie::DekoratorNasycenie (
    std::unique_ptr< Sygnal > k,
    double max)
```

Construct a new saturation decorator.

Parameters

	<i>k</i>	Signal to decorate
	<i>max</i>	Maximum allowed value

4.2.3 Member Function Documentation

4.2.3.1 klonuj()

```
std::unique_ptr< Signal > DekoratorNasycenie::klonuj () const [override], [virtual]
```

Clone the signal object.

Returns

std::unique_ptr<Signal> Copy of the signal

Implements [Dekorator](#).

4.2.3.2 serializuj()

```
void DekoratorNasycenie::serializuj (  
    std::ostream & out) const [override], [virtual]
```

Serialize signal to output stream.

Parameters

	<i>out</i>	Output stream
--	------------	---------------

Implements [Dekorator](#).

4.2.3.3 symuluj()

```
double DekoratorNasycenie::symuluj (  
    int t) [override], [virtual]
```

Simulate signal value at time t.

Parameters

	<i>t</i>	Time step
--	----------	-----------

Returns

double Signal value

Implements [Dekorator](#).

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/[DekoratorNasycenie.h](#)
- Programowanie_Objektowe_04/[DekoratorNasycenie.cpp](#)

4.3 DekoratorProstokat Class Reference

Rectangular wave signal decorator.

```
#include <DekoratorProstokat.h>
```

Inheritance diagram for DekoratorProstokat:

Collaboration diagram for DekoratorProstokat:

Public Member Functions

- [DekoratorProstokat](#) (std::unique_ptr< [Sygnal](#) > k, double amp, int okr, double wyp)
Construct a new rectangular wave decorator.
- double [symuluj](#) (int t) override
Simulate signal value at time t.
- void [serializuj](#) (std::ostream &out) const override
Serialize signal to output stream.
- std::unique_ptr< [Sygnal](#) > [klonuj](#) () const override
Clone the signal object.

Public Member Functions inherited from [Dekorator](#)

- [Dekorator](#) (std::unique_ptr< [Sygnal](#) > k)
Construct a new [Dekorator](#) object.

Additional Inherited Members

Protected Attributes inherited from [Dekorator](#)

- std::unique_ptr< [Sygnal](#) > [komponent](#)

4.3.1 Detailed Description

Rectangular wave signal decorator.

4.3.2 Constructor & Destructor Documentation

4.3.2.1 DekoratorProstokat()

```
DekoratorProstokat::DekoratorProstokat (  
    std::unique_ptr< Sygnal > k,  
    double amp,  
    int okr,  
    double wyp)
```

Construct a new rectangular wave decorator.

Parameters

	<i>k</i>	Signal to decorate
	<i>amp</i>	Amplitude
	<i>okr</i>	Period
	<i>wyp</i>	Duty cycle

4.3.3 Member Function Documentation**4.3.3.1 klonuj()**

```
std::unique_ptr< Signal > DekoratorProstokat::klonuj () const [override], [virtual]
```

Clone the signal object.

Returns

std::unique_ptr<Signal> Copy of the signal

Implements [Dekorator](#).

4.3.3.2 serializuj()

```
void DekoratorProstokat::serializuj (
    std::ostream & out) const [override], [virtual]
```

Serialize signal to output stream.

Parameters

	<i>out</i>	Output stream
--	------------	---------------

Implements [Dekorator](#).

4.3.3.3 symuluj()

```
double DekoratorProstokat::symuluj (
    int t) [override], [virtual]
```

Simulate signal value at time t.

Parameters

	<i>t</i>	Time step
--	----------	-----------

Returns

double Signal value

Implements [Dekorator](#).

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/[DekoratorProstokat.h](#)
- Programowanie_Objektowe_04/[DekoratorProstokat.cpp](#)

4.4 DekoratorSinus Class Reference

Sinusoidal wave signal decorator.

```
#include <DekoratorSinus.h>
```

Inheritance diagram for DekoratorSinus:

Collaboration diagram for DekoratorSinus:

Public Member Functions

- [DekoratorSinus](#) (std::unique_ptr< [Sygnal](#) > k, double amp, double czest)
Construct a new sinusoidal decorator.
- double [symuluj](#) (int t) override
Simulate signal value at time t.
- void [serializuj](#) (std::ostream &out) const override
Serialize signal to output stream.
- std::unique_ptr< [Sygnal](#) > [klonuj](#) () const override
Clone the signal object.

Public Member Functions inherited from [Dekorator](#)

- [Dekorator](#) (std::unique_ptr< [Sygnal](#) > k)
Construct a new [Dekorator](#) object.

Additional Inherited Members

Protected Attributes inherited from [Dekorator](#)

- std::unique_ptr< [Sygnal](#) > [komponent](#)

4.4.1 Detailed Description

Sinusoidal wave signal decorator.

4.4.2 Constructor & Destructor Documentation

4.4.2.1 DekoratorSinus()

```
DekoratorSinus::DekoratorSinus (  
    std::unique_ptr< Sygnal > k,  
    double amp,  
    double czest)
```

Construct a new sinusoidal decorator.

Parameters

	<i>k</i>	Signal to decorate
	<i>amp</i>	Amplitude
	<i>czest</i>	Frequency

4.4.3 Member Function Documentation

4.4.3.1 klonuj()

```
std::unique_ptr< Sygnal > DekoratorSinus::klonuj () const [override], [virtual]
```

Clone the signal object.

Returns

std::unique_ptr<Sygnal> Copy of the signal

Implements [Dekorator](#).

4.4.3.2 serializuj()

```
void DekoratorSinus::serializuj (
    std::ostream & out) const [override], [virtual]
```

Serialize signal to output stream.

Parameters

	<i>out</i>	Output stream
--	------------	---------------

Implements [Dekorator](#).

4.4.3.3 symuluj()

```
double DekoratorSinus::symuluj (
    int t) [override], [virtual]
```

Simulate signal value at time t.

Parameters

	<i>t</i>	Time step
--	----------	-----------

Returns

double Signal value

Implements [Dekorator](#).

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/[DekoratorSinus.h](#)
- Programowanie_Objektowe_04/[DekoratorSinus.cpp](#)

4.5 DekoratorSzumBialy Class Reference

White noise signal decorator.

```
#include <DekoratorSzumBialy.h>
```

Inheritance diagram for DekoratorSzumBialy:

Collaboration diagram for DekoratorSzumBialy:

Public Member Functions

- [DekoratorSzumBialy](#) (std::unique_ptr< [Sygnal](#) > k, double odch)
Construct a new white noise decorator.
- double [symuluj](#) (int t) override
Simulate signal value at time t.
- void [serializuj](#) (std::ostream &out) const override
Serialize signal to output stream.
- std::unique_ptr< [Sygnal](#) > [klonuj](#) () const override
Clone the signal object.

Public Member Functions inherited from [Dekorator](#)

- [Dekorator](#) (std::unique_ptr< [Sygnal](#) > k)
Construct a new [Dekorator](#) object.

Additional Inherited Members

Protected Attributes inherited from [Dekorator](#)

- std::unique_ptr< [Sygnal](#) > [komponent](#)

4.5.1 Detailed Description

White noise signal decorator.

4.5.2 Constructor & Destructor Documentation

4.5.2.1 DekoratorSzumBialy()

```
DekoratorSzumBialy::DekoratorSzumBialy (  
    std::unique_ptr< Sygnal > k,  
    double odch)
```

Construct a new white noise decorator.

Parameters

	<i>k</i>	Signal to decorate
	<i>odch</i>	Noise standard deviation

4.5.3 Member Function Documentation

4.5.3.1 klonuj()

```
std::unique_ptr< Signal > DekoratorSzumBialy::klonuj () const [override], [virtual]
```

Clone the signal object.

Returns

std::unique_ptr<Signal> Copy of the signal

Implements [Dekorator](#).

4.5.3.2 serializuj()

```
void DekoratorSzumBialy::serializuj (  
    std::ostream & out) const [override], [virtual]
```

Serialize signal to output stream.

Parameters

	<i>out</i>	Output stream
--	------------	---------------

Implements [Dekorator](#).

4.5.3.3 symuluj()

```
double DekoratorSzumBialy::symuluj (  
    int t) [override], [virtual]
```

Simulate signal value at time t.

Parameters

	<i>t</i>	Time step
--	----------	-----------

Returns

double Signal value

Implements [Dekorator](#).

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/[DekoratorSzumBialy.h](#)
- Programowanie_Objektowe_04/[DekoratorSzumBialy.cpp](#)

4.6 DekoratorTrojkat Class Reference

Triangular wave signal decorator.

```
#include <DekoratorTrojkat.h>
```

Inheritance diagram for DekoratorTrojkat:

Collaboration diagram for DekoratorTrojkat:

Public Member Functions

- [DekoratorTrojkat](#) (std::unique_ptr< [Sygnal](#) > k, double amp, int okr, double wyp)
Construct a new triangular wave decorator.
- double [symuluj](#) (int t) override
Simulate signal value at time t.
- void [serializuj](#) (std::ostream &out) const override
Serialize signal to output stream.
- std::unique_ptr< [Sygnal](#) > [klonuj](#) () const override
Clone the signal object.

Public Member Functions inherited from [Dekorator](#)

- [Dekorator](#) (std::unique_ptr< [Sygnal](#) > k)
Construct a new [Dekorator](#) object.

Additional Inherited Members

Protected Attributes inherited from [Dekorator](#)

- std::unique_ptr< [Sygnal](#) > [komponent](#)

4.6.1 Detailed Description

Triangular wave signal decorator.

4.6.2 Constructor & Destructor Documentation

4.6.2.1 DekoratorTrojkat()

```
DekoratorTrojkat::DekoratorTrojkat (
    std::unique_ptr< Sygnal > k,
    double amp,
    int okr,
    double wyp)
```

Construct a new triangular wave decorator.

Parameters

	<i>k</i>	Signal to decorate
	<i>amp</i>	Amplitude
	<i>okr</i>	Period
	<i>wyp</i>	Symmetry ratio

4.6.3 Member Function Documentation

4.6.3.1 klonuj()

```
std::unique_ptr< Signal > DekoratorTrojkat::klonuj () const [override], [virtual]
```

Clone the signal object.

Returns

std::unique_ptr<Signal> Copy of the signal

Implements [Dekorator](#).

4.6.3.2 serializuj()

```
void DekoratorTrojkat::serializuj (  
    std::ostream & out) const [override], [virtual]
```

Serialize signal to output stream.

Parameters

	<i>out</i>	Output stream
--	------------	---------------

Implements [Dekorator](#).

4.6.3.3 symuluj()

```
double DekoratorTrojkat::symuluj (  
    int t) [override], [virtual]
```

Simulate signal value at time t.

Parameters

	<i>t</i>	Time step
--	----------	-----------

Returns

double Signal value

Implements [Dekorator](#).

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/[DekoratorTrojkat.h](#)
- Programowanie_Objektowe_04/[DekoratorTrojkat.cpp](#)

4.7 Konfiguracja Class Reference

Holds configuration settings for the control system simulation.

```
#include <Konfiguracja.h>
```

Public Member Functions

- **Konfiguracja** ()
Default constructor.
- void **wczytaj** (const std::string &nazwaPliku)
Load configuration from a file.
- void **zapisz** (const std::string &nazwaPliku) const
Save configuration to a file.

Setters

Methods used to configure the simulation parameters

- void **ustawPetle** (std::unique_ptr< [PetlaUAR](#) > nowaPetla)
- void **ustawGenerator** (std::unique_ptr< [Sygnal](#) > nowyGenerator)
- void **ustawLiczbeKrokow** (int kroki)
- void **ustawZamknieta** (bool stan)

Getters

Methods used to retrieve the current configuration state

- [PetlaUAR](#) * **pobierzPetle** () const
- [Sygnal](#) * **pobierzGenerator** () const
- int **pobierzLiczbeKrokow** () const
- bool **czyZamknieta** () const

4.7.1 Detailed Description

Holds configuration settings for the control system simulation.

4.7.2 Member Function Documentation

4.7.2.1 wczytaj()

```
void Konfiguracja::wczytaj (
    const std::string & nazwaPliku)
```

Load configuration from a file.

Parameters

<code>nazwaPliku</code>	Name of the file to load
-------------------------	--------------------------

4.7.2.2 zapisz()

```
void Konfiguracja::zapisz (
    const std::string & nazwaPliku) const
```

Save configuration to a file.

Parameters

<code>nazwaPliku</code>	Name of the file to save to
-------------------------	-----------------------------

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/[Konfiguracja.h](#)
- Programowanie_Objektowe_04/[Konfiguracja.cpp](#)

4.8 ModelARX Class Reference

ARX (AutoRegressive with eXogenous input) model implementation.

```
#include <ModelARX.h>
```

Inheritance diagram for ModelARX:

Collaboration diagram for ModelARX:

Public Member Functions

- [ModelARX](#) (std::vector< double > a, std::vector< double > b, unsigned k, double zmienna)
Constructor with parameters.
- [ModelARX](#) (const std::string &nazwaPliku)
Constructor from configuration file.
- double [symuluj](#) (double u) override
Simulate system output for given input.
- void [serializuj](#) (const std::string &nazwaPliku) const
Serialize model parameters to file.

4.8.1 Detailed Description

ARX (AutoRegressive with eXogenous input) model implementation.

This class implements a discrete-time ARX model with optional noise disturbance. The model follows the equation: $y(k) + a_1*y(k-1) + \dots + a_n*y(k-n) = b_0*u(k-d) + \dots + b_m*u(k-d-m) + e(k)$ where $e(k)$ is optional Gaussian noise.

4.8.2 Constructor & Destructor Documentation

4.8.2.1 ModelARX() [1/2]

```
ModelARX::ModelARX (
    std::vector< double > a,
    std::vector< double > b,
    unsigned k,
    double zmienna)
```

Constructor with parameters.

Parameters

<code>a</code>	Vector of autoregressive coefficients (a1 to an)
<code>b</code>	Vector of input coefficients (b0 to bm)
<code>k</code>	Input delay (discrete time steps)
<code>zmienna</code>	Noise standard deviation (set to 0 for no noise)

4.8.2.2 ModelARX() [2/2]

```
ModelARX::ModelARX (
    const std::string & nazwaPliku)
```

Constructor from configuration file.

Parameters

<code>nazwaPliku</code>	Path to configuration file containing model parameters
-------------------------	--

Exceptions

<code>std::runtime_error</code>	if file cannot be read or is malformed
---------------------------------	--

4.8.3 Member Function Documentation

4.8.3.1 serializuj()

```
void ModelARX::serializuj (
    const std::string & nazwaPliku) const
```

Serialize model parameters to file.

Parameters

<code>nazwaPliku</code>	Output file path
-------------------------	------------------

Exceptions

<code>std::runtime_error</code>	if file cannot be written
---------------------------------	---------------------------

4.8.3.2 symuluj()

```
double ModelARX::symuluj (
    double u) [override], [virtual]
```

Simulate system output for given input.

Parameters

u	Input value
-----	-------------

Returns

double Output value

Implements [ObiektSISO](#).

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/ModelARX.h
- Programowanie_Objektowe_04/ModelARX.cpp

4.9 Nasycenie Class Reference

Inheritance diagram for Nasycenie:

Collaboration diagram for Nasycenie:

Public Member Functions

- **Nasycenie** (std::unique_ptr< [Sygnal](#) > s, double max)
- double **generuj** (int t) override
- void **serializuj** (std::ostream &out) const override
Serialize signal to output stream.

Public Member Functions inherited from [Sygnal](#)

- virtual double **symuluj** (int t)=0
Simulate signal value at time t.
- virtual std::unique_ptr< [Sygnal](#) > **klonuj** () const =0
Clone the signal object.

4.9.1 Member Function Documentation

4.9.1.1 serializuj()

```
void Nasycenie::serializuj (
    std::ostream & out) const [override], [virtual]
```

Serialize signal to output stream.

Parameters

<code>out</code>	Output stream
------------------	---------------

Implements [Sygnal](#).

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/Nasycenie.h
- Programowanie_Objektowe_04/Nasycenie.cpp

4.10 ObiektSISO Class Reference

Abstract SISO system.

```
#include <ObiektSISO.h>
```

Inheritance diagram for ObiektSISO:

Public Member Functions

- virtual double [symuluj](#) (double u)=0
Simulate system output for given input.

4.10.1 Detailed Description

Abstract SISO system.

4.10.2 Member Function Documentation

4.10.2.1 [symuluj\(\)](#)

```
virtual double ObiektSISO::symuluj (
    double u) [pure virtual]
```

Simulate system output for given input.

Parameters

<code>u</code>	Input value
----------------	-------------

Returns

double Output value

Implemented in [ModelARX](#), [ObiektStatyczny](#), [PetlaUAR](#), and [RegulatorPID](#).

The documentation for this class was generated from the following file:

- Programowanie_Objektowe_04/[ObiektSISO.h](#)

4.11 ObiektStatyczny Class Reference

Represents a static SISO object with output range limits.

```
#include <ObiektStatyczny.h>
```

Inheritance diagram for ObiektStatyczny:

Collaboration diagram for ObiektStatyczny:

Public Member Functions

- [ObiektStatyczny](#) (double d, double g)
Construct static object with given limits.
- [ObiektStatyczny](#) (const std::string &plik)
Construct static object from file.
- double [symuluj](#) (double x) override
Simulate system output for given input.
- void [serializuj](#) (const std::string &plik) const
Serialize the object to a file.

4.11.1 Detailed Description

Represents a static SISO object with output range limits.

4.11.2 Constructor & Destructor Documentation

4.11.2.1 ObiektStatyczny() [1/2]

```
ObiektStatyczny::ObiektStatyczny (
    double d,
    double g)
```

Construct static object with given limits.

Parameters

d	Lower bound
g	Upper bound

4.11.2.2 ObiektStatyczny() [2/2]

```
ObiektStatyczny::ObiektStatyczny (
    const std::string & plik)
```

Construct static object from file.

Parameters

<code>plik</code>	File containing object data
-------------------	-----------------------------

4.11.3 Member Function Documentation

4.11.3.1 `serializuj()`

```
void ObiektStatyczny::serializuj (
    const std::string & plik) const
```

Serialize the object to a file.

Parameters

<code>plik</code>	Target file path
-------------------	------------------

4.11.3.2 `symuluj()`

```
double ObiektStatyczny::symuluj (
    double u) [override], [virtual]
```

Simulate system output for given input.

Parameters

<code>u</code>	Input value
----------------	-------------

Returns

double Output value

Implements [ObiektSISO](#).

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/[ObiektStatyczny.h](#)
- Programowanie_Objektowe_04/ObiektStatyczny.cpp

4.12 PetlaUAR Class Reference

Represents a control loop composed of multiple SISO components.

```
#include <PetlaUAR.h>
```

Inheritance diagram for PetlaUAR:

Collaboration diagram for PetlaUAR:

Public Member Functions

- double [symuluj](#) (double x) override
Simulate the loop response for given input.
- void [ustawZamknietą](#) (bool stan)
Set loop mode to open or closed.
- void [dodajKomponent](#) (std::unique_ptr< [ObiektSISO](#) > obiekt, const std::string &typ)
Add a new component to the loop.
- void [dodajKomponentNaPozycji](#) (std::unique_ptr< [ObiektSISO](#) > obiekt, size_t index, const std::string &typ)
Insert a component at a specific position.
- void [usunKomponent](#) (size_t index)
Remove a component from the loop.
- void [serializuj](#) (const std::string &nazwaPliku) const
Serialize the loop to a file.
- void [deserializuj](#) (const std::string &nazwaPliku)
Deserialize the loop from a file.
- std::vector< std::string > [pobierzTypyKomponentow](#) () const
Get list of component types.

4.12.1 Detailed Description

Represents a control loop composed of multiple SISO components.

Supports both open-loop and closed-loop operation, and allows dynamic composition of components

4.12.2 Member Function Documentation

4.12.2.1 deserializuj()

```
void PetlaUAR::deserializuj (
    const std::string & nazwaPliku)
```

Deserialize the loop from a file.

Parameters

<code>nazwaPliku</code>	Source file path
-------------------------	------------------

4.12.2.2 dodajKomponent()

```
void PetlaUAR::dodajKomponent (
    std::unique_ptr< ObiektSISO > obiekt,
    const std::string & typ)
```

Add a new component to the loop.

Parameters

	<i>obiekt</i>	Component to add
	<i>typ</i>	Type identifier of the component

4.12.2.3 dodajKomponentNaPozycji()

```
void PetlaUAR::dodajKomponentNaPozycji (
    std::unique_ptr< ObiektSISO > obiekt,
    size_t index,
    const std::string & typ)
```

Insert a component at a specific position.

Parameters

	<i>obiekt</i>	Component to add
	<i>index</i>	Position in the loop
	<i>typ</i>	Type identifier of the component

4.12.2.4 pobierzTypyKomponentow()

```
std::vector< std::string > PetlaUAR::pobierzTypyKomponentow () const
```

Get list of component types.

Returns

`std::vector<std::string>` Vector of type names

4.12.2.5 serializuj()

```
void PetlaUAR::serializuj (
    const std::string & nazwaPliku) const
```

Serialize the loop to a file.

Parameters

	<i>nazwaPliku</i>	Target file path
--	-------------------	------------------

4.12.2.6 symuluj()

```
double PetlaUAR::symuluj (
    double x) [override], [virtual]
```

Simulate the loop response for given input.

Parameters

☐	<i>x</i>	Input value
--------------------------	----------	-------------

Returns

double Output value

Implements [ObiektSISO](#).

4.12.2.7 ustawZamknieta()

```
void PetlaUAR::ustawZamknieta (  
    bool stan)
```

Set loop mode to open or closed.

Parameters

☐	<i>stan</i>	True for closed-loop, false for open-loop
--------------------------	-------------	---

4.12.2.8 usunKomponent()

```
void PetlaUAR::usunKomponent (  
    size_t index)
```

Remove a component from the loop.

Parameters

☐	<i>index</i>	Index of the component to remove
--------------------------	--------------	----------------------------------

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/[PetlaUAR.h](#)
- Programowanie_Objektowe_04/[PetlaUAR.cpp](#)

4.13 Prostokat Class Reference

Inheritance diagram for Prostokat:

Collaboration diagram for Prostokat:

Public Member Functions

- **Prostokat** (double a, int o, double w)
- double **generuj** (int t) override
- void **serializuj** (std::ostream &out) const override
Serialize signal to output stream.

Public Member Functions inherited from [Sygnal](#)

- virtual double **symuluj** (int t)=0
Simulate signal value at time t.
- virtual std::unique_ptr< [Sygnal](#) > **klonuj** () const =0
Clone the signal object.

4.13.1 Member Function Documentation

4.13.1.1 serializuj()

```
void Prostokat::serializuj (
    std::ostream & out) const    [override], [virtual]
```

Serialize signal to output stream.

Parameters

<code>out</code>	Output stream
------------------	---------------

Implements [Sygnal](#).

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/Prostokat.h
- Programowanie_Objektowe_04/Prostokat.cpp

4.14 RegulatorPID Class Reference

PID controller with configurable modes (P/PI/PID)

```
#include <RegulatorPID.h>
```

Inheritance diagram for RegulatorPID:

Collaboration diagram for RegulatorPID:

Public Member Functions

- [RegulatorPID](#) (double k)
P-only controller.
- [RegulatorPID](#) (double k , double T_i)
PI controller.
- [RegulatorPID](#) (double k , double T_i , double T_d)
Full PID controller.
- double [symuluj](#) (double $uchyb$) override
Simulate system output for given input.
- **RegulatorPID** (const std::string &plik)
Konstruktor z pliku.
- void **serializuj** (const std::string &plik) const
Zapis do pliku.

4.14.1 Detailed Description

PID controller with configurable modes (P/PI/PID)

4.14.2 Constructor & Destructor Documentation

4.14.2.1 [RegulatorPID\(\)](#) [1/3]

```
RegulatorPID::RegulatorPID (
    double  $k$ ) [explicit]
```

P-only controller.

Parameters

k	Proportional gain
-----	-------------------

4.14.2.2 [RegulatorPID\(\)](#) [2/3]

```
RegulatorPID::RegulatorPID (
    double  $k$ ,
    double  $T_i$ )
```

PI controller.

Parameters

k	Proportional gain
T_i	Integral time

4.14.2.3 [RegulatorPID\(\)](#) [3/3]

```
RegulatorPID::RegulatorPID (
    double  $k$ ,
    double  $T_i$ ,
    double  $T_d$ )
```

Full PID controller.

Parameters

k	Proportional gain
T_i	Integral time
T_d	Derivative time

4.14.3 Member Function Documentation

4.14.3.1 `symuluj()`

```
double RegulatorPID::symuluj (
    double u) [override], [virtual]
```

Simulate system output for given input.

Parameters

u	Input value
-----	-------------

Returns

double Output value

Implements [ObiektSISO](#).

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/[RegulatorPID.h](#)
- Programowanie_Objektowe_04/RegulatorPID.cpp

4.15 Sinus Class Reference

Inheritance diagram for Sinus:

Collaboration diagram for Sinus:

Public Member Functions

- **Sinus** (double a, double f)
- double **generuj** (int t) override
- void [serializuj](#) (std::ostream &out) const override
Serialize signal to output stream.

Public Member Functions inherited from [Sygnal](#)

- virtual double [symuluj](#) (int t)=0
Simulate signal value at time t.
- virtual std::unique_ptr< [Sygnal](#) > [klonuj](#) () const =0
Clone the signal object.

4.15.1 Member Function Documentation

4.15.1.1 serializuj()

```
void Sinus::serializuj (
    std::ostream & out) const [override], [virtual]
```

Serialize signal to output stream.

Parameters

<code>out</code>	Output stream
------------------	---------------

Implements [Sygnal](#).

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/Sinus.h
- Programowanie_Objektowe_04/Sinus.cpp

4.16 Stala Class Reference

Inheritance diagram for Stala:

Collaboration diagram for Stala:

Public Member Functions

- **Stala** (double w)
- double **generuj** (int t) override
- void [serializuj](#) (std::ostream &out) const override
Serialize signal to output stream.

Public Member Functions inherited from [Sygnal](#)

- virtual double [symuluj](#) (int t)=0
Simulate signal value at time t.
- virtual std::unique_ptr< [Sygnal](#) > [klonuj](#) () const =0
Clone the signal object.

4.16.1 Member Function Documentation

4.16.1.1 serializuj()

```
void Stala::serializuj (
    std::ostream & out) const [override], [virtual]
```

Serialize signal to output stream.

Parameters

<code>out</code>	Output stream
------------------	---------------

Implements [Sygnal](#).

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/Stala.h
- Programowanie_Objektowe_04/Stala.cpp

4.17 Sygnal Class Reference

Interface for signal generators.

```
#include <Sygnal.h>
```

Inheritance diagram for Sygnal:

Public Member Functions

- virtual double [symuluj](#) (int t)=0
Simulate signal value at time t.
- virtual void [serializuj](#) (std::ostream &out) const =0
Serialize signal to output stream.
- virtual std::unique_ptr< [Sygnal](#) > [klonuj](#) () const =0
Clone the signal object.

4.17.1 Detailed Description

Interface for signal generators.

4.17.2 Member Function Documentation

4.17.2.1 klonuj()

```
virtual std::unique_ptr< Sygnal > Sygnal::klonuj () const [pure virtual]
```

Clone the signal object.

Returns

std::unique_ptr<Sygnal> Copy of the signal

Implemented in [Dekorator](#), [DekoratorNasycenie](#), [DekoratorProstokat](#), [DekoratorSinus](#), [DekoratorSzumBialy](#), [DekoratorTrojkat](#), and [SygnalStaly](#).

4.17.2.2 serializuj()

```
virtual void Sygnal::serializuj (  
    std::ostream & out) const [pure virtual]
```

Serialize signal to output stream.

Parameters

<code>out</code>	Output stream
------------------	---------------

Implemented in [Dekorator](#), [DekoratorNasycenie](#), [DekoratorProstokat](#), [DekoratorSinus](#), [DekoratorSzumBialy](#), [DekoratorTrojkat](#), [Nasycenie](#), [Prostokat](#), [Sinus](#), [Stala](#), [SygnalStaly](#), [SygnalZlozony](#), [SzumBialy](#), and [Trojkat](#).

4.17.2.3 symuluj()

```
virtual double Sygnal::symuluj (
    int t) [pure virtual]
```

Simulate signal value at time t.

Parameters

<code>t</code>	Time step
----------------	-----------

Returns

double Signal value

Implemented in [Dekorator](#), [DekoratorNasycenie](#), [DekoratorProstokat](#), [DekoratorSinus](#), [DekoratorSzumBialy](#), [DekoratorTrojkat](#), and [SygnalStaly](#).

The documentation for this class was generated from the following file:

- Programowanie_Objektowe_04/[Sygnal.h](#)

4.18 SygnalStaly Class Reference

Constant value signal.

```
#include <SygnalStaly.h>
```

Inheritance diagram for SygnalStaly:

Collaboration diagram for SygnalStaly:

Public Member Functions

- [SygnalStaly](#) (double w=0.0)
Construct a new constant signal.
- double [symuluj](#) (int t) override
Simulate signal value at time t.
- void [serializuj](#) (std::ostream &out) const override
Serialize signal to output stream.
- std::unique_ptr< [Sygnal](#) > [klonuj](#) () const override
Clone the signal object.

4.18.1 Detailed Description

Constant value signal.

4.18.2 Constructor & Destructor Documentation

4.18.2.1 SygnalStaly()

```
SygnalStaly::SygnalStaly (
    double w = 0.0) [explicit]
```

Construct a new constant signal.

Parameters

<code>w</code>	Signal value (default 0.0)
----------------	----------------------------

4.18.3 Member Function Documentation

4.18.3.1 klonuj()

```
std::unique_ptr< Sygnal > SygnalStaly::klonuj () const [override], [virtual]
```

Clone the signal object.

Returns

std::unique_ptr<Sygnal> Copy of the signal

Implements [Sygnal](#).

4.18.3.2 serializuj()

```
void SygnalStaly::serializuj (
    std::ostream & out) const [override], [virtual]
```

Serialize signal to output stream.

Parameters

<code>out</code>	Output stream
------------------	---------------

Implements [Sygnal](#).

4.18.3.3 symuluj()

```
double SygnalStaly::symuluj (
    int t) [override], [virtual]
```

Simulate signal value at time t.

Parameters

t	Time step
-----	-----------

Returns

double Signal value

Implements [Sygnal](#).

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/[SygnalStaly.h](#)
- Programowanie_Objektowe_04/SygnalStaly.cpp

4.19 SygnalZlozony Class Reference

Inheritance diagram for SygnalZlozony:

Collaboration diagram for SygnalZlozony:

Public Member Functions

- **SygnalZlozony** (std::vector< std::unique_ptr< [Sygnal](#) > > s)
- double **generuj** (int t) override
- void **serializuj** (std::ostream &out) const override
Serialize signal to output stream.

Public Member Functions inherited from [Sygnal](#)

- virtual double **symuluj** (int t)=0
Simulate signal value at time t.
- virtual std::unique_ptr< [Sygnal](#) > **klonuj** () const =0
Clone the signal object.

4.19.1 Member Function Documentation

4.19.1.1 serializuj()

```
void SygnalZlozony::serializuj (
    std::ostream & out) const [override], [virtual]
```

Serialize signal to output stream.

Parameters

<code>out</code>	Output stream
------------------	---------------

Implements [Sygnal](#).

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/SygnalZlozony.h
- Programowanie_Objektowe_04/SygnalZlozony.cpp

4.20 SzumBialy Class Reference

Inheritance diagram for SzumBialy:

Collaboration diagram for SzumBialy:

Public Member Functions

- **SzumBialy** (double srednia, double odchylenie)
- double **generuj** (int t) override
- void **serializuj** (std::ostream &out) const override
Serialize signal to output stream.

Public Member Functions inherited from [Sygnal](#)

- virtual double **symuluj** (int t)=0
Simulate signal value at time t.
- virtual std::unique_ptr< [Sygnal](#) > **klonuj** () const =0
Clone the signal object.

4.20.1 Member Function Documentation

4.20.1.1 serializuj()

```
void SzumBialy::serializuj (  
    std::ostream & out) const [override], [virtual]
```

Serialize signal to output stream.

Parameters

<code>out</code>	Output stream
------------------	---------------

Implements [Sygnal](#).

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/SzumBialy.h
- Programowanie_Objektowe_04/SzumBialy.cpp

4.21 Trojkat Class Reference

Inheritance diagram for Trojkat:

Collaboration diagram for Trojkat:

Public Member Functions

- **Trojkat** (double a, int o, double w)
- double **generuj** (int t) override
- void **serializuj** (std::ostream &out) const override
Serialize signal to output stream.

Public Member Functions inherited from [Sygnal](#)

- virtual double **symuluj** (int t)=0
Simulate signal value at time t.
- virtual std::unique_ptr< [Sygnal](#) > **klonuj** () const =0
Clone the signal object.

4.21.1 Member Function Documentation

4.21.1.1 serializuj()

```
void Trojkat::serializuj (  
    std::ostream & out) const    [override], [virtual]
```

Serialize signal to output stream.

Parameters

<code>out</code>	Output stream
------------------	---------------

Implements [Sygnal](#).

The documentation for this class was generated from the following files:

- Programowanie_Objektowe_04/Trojkat.h
- Programowanie_Objektowe_04/Trojkat.cpp

Chapter 5

File Documentation

5.1 Programowanie_Objektowe_04/Dekorator.h File Reference

Base decorator class for signal processing (Decorator Pattern)

```
#include "Sygnal.h"
#include <memory>
Include dependency graph for Dekorator.h:
```

5.2 Dekorator.h

[Go to the documentation of this file.](#)

```
00001
00005 #pragma once
00006 #include "Sygnal.h"
00007 #include <memory>
00008
00013 class Dekorator :
00014     public Sygnal
00015 {
00016 protected:
00017     std::unique_ptr<Sygnal> komponent;
00018 public:
00023     Dekorator(std::unique_ptr<Sygnal> k);
00024
00025     double symuluj(int t) override = 0;
00026     void serializuj(std::ostream& out) const override = 0;
00027     std::unique_ptr<Sygnal> klonuj() const override = 0;
00028 };
```

5.3 Programowanie_Objektowe_04/DekoratorNasycenie.h File Reference

Decorator that applies saturation (clipping) to a signal.

```
#include "Dekorator.h"
Include dependency graph for DekoratorNasycenie.h:
```

Classes

- class [DekoratorNasycenie](#)
Saturation decorator limits signal amplitude to a maximum value.

5.3.1 Detailed Description

Decorator that applies saturation (clipping) to a signal.

5.4 DekoratorNasycenie.h

[Go to the documentation of this file.](#)

```
00001
00005 #pragma once
00006 #include "Dekorator.h"
00007
00011 class DekoratorNasycenie :
00012     public Dekorator
00013 {
00014     double max_wartosc;
00015 public:
00021     DekoratorNasycenie(std::unique_ptr<Sygnal> k, double max);
00022
00023     double symuluj(int t) override;
00024     void serializuj(std::ostream& out) const override;
00025     std::unique_ptr<Sygnal> klonuj() const override;
00026 };
```

5.5 Programowanie_Obiektowe_04/DekoratorProstokat.h File Reference

Decorator that applies rectangular wave modulation.

```
#include "Dekorator.h"
```

Include dependency graph for DekoratorProstokat.h:

Classes

- class [DekoratorProstokat](#)
Rectangular wave signal decorator.

5.5.1 Detailed Description

Decorator that applies rectangular wave modulation.

5.6 DekoratorProstokat.h

[Go to the documentation of this file.](#)

```
00001
00005 #pragma once
00006 #include "Dekorator.h"
00007
00011 class DekoratorProstokat :
00012     public Dekorator
00013 {
00014     double amplituda;
00015     int okres;
00016     double wypelnienie;
00017 public:
00025     DekoratorProstokat(std::unique_ptr<Sygnal> k, double amp, int okr, double wyp);
00026
00027     double symuluj(int t) override;
00028     void serializuj(std::ostream& out) const override;
00029     std::unique_ptr<Sygnal> klonuj() const override;
00030 };
```

5.7 Programowanie_Objektowe_04/DekoratorSinus.h File Reference

Decorator that applies sinusoidal modulation.

```
#include "Dekorator.h"
```

```
#include <cmath>
```

Include dependency graph for DekoratorSinus.h:

Classes

- class [DekoratorSinus](#)
Sinusoidal wave signal decorator.

5.7.1 Detailed Description

Decorator that applies sinusoidal modulation.

5.8 DekoratorSinus.h

[Go to the documentation of this file.](#)

```
00001
00005 #pragma once
00006 #include "Dekorator.h"
00007 #include <cmath>
00008
00012 class DekoratorSinus :
00013     public Dekorator
00014 {
00015     double amplituda, czestotliwosc;
00016     double y_prev1, y_prev2;
00017     int t_prev;
00018 public:
00025     DekoratorSinus(std::unique_ptr<Sygnal> k, double amp, double czest);
00026
00027     double symuluj(int t) override;
00028     void serializuj(std::ostream& out) const override;
00029     std::unique_ptr<Sygnal> klonuj() const override;
00030 };
```

5.9 Programowanie_Objektowe_04/DekoratorSzumBialy.h File Reference

Decorator that adds white noise to a signal.

```
#include "Dekorator.h"
```

```
#include <random>
```

Include dependency graph for DekoratorSzumBialy.h:

Classes

- class [DekoratorSzumBialy](#)
White noise signal decorator.

5.9.1 Detailed Description

Decorator that adds white noise to a signal.

5.10 DekoratorSzumBialy.h

[Go to the documentation of this file.](#)

```
00001
00005 #pragma once
00006 #include "Dekorator.h"
00007 #include <random>
00008
00012 class DekoratorSzumBialy :
00013     public Dekorator
00014 {
00015     double odchylenie;
00016     std::default_random_engine generator;
00017     std::normal_distribution<double> rozklad;
00018 public:
00024     DekoratorSzumBialy(std::unique_ptr<Sygnal> k, double odch);
00025
00026     double symuluj(int t) override;
00027     void serializuj(std::ostream& out) const override;
00028     std::unique_ptr<Sygnal> klonuj() const override;
00029 };
```

5.11 Programowanie_Objektowe_04/DekoratorTrojkat.h File Reference

Decorator that applies triangular wave modulation.

```
#include "Dekorator.h"
```

Include dependency graph for DekoratorTrojkat.h:

Classes

- class [DekoratorTrojkat](#)
Triangular wave signal decorator.

5.11.1 Detailed Description

Decorator that applies triangular wave modulation.

5.12 DekoratorTrojkat.h

[Go to the documentation of this file.](#)

```
00001
00005 #pragma once
00006 #include "Dekorator.h"
00007
00011 class DekoratorTrojkat :
00012     public Dekorator
00013 {
00014     double amplituda;
00015     int okres;
00016     double wypelnienie;
00017 public:
00025     DekoratorTrojkat(std::unique_ptr<Sygnal> k, double amp, int okr, double wyp);
00026
00027     double symuluj(int t) override;
00028     void serializuj(std::ostream& out) const override;
00029     std::unique_ptr<Sygnal> klonuj() const override;
00030 };
```

5.13 Deserializacja.h

```
00001 #pragma once
00002 #include "Sygnal.h"
00003 #include <memory>
00004 #include <istream>
00005
00006 std::unique_ptr<Sygnal> deserializujSygnal(std::istream& in);
```

5.14 Programowanie_Objektowe_04/Konfiguracja.h File Reference

Configuration class for the control loop simulation.

```
#include "PetlaUAR.h"
#include "Sygnal.h"
#include <memory>
#include <string>
Include dependency graph for Konfiguracja.h:
```

Classes

- class [Konfiguracja](#)
Holds configuration settings for the control system simulation.

5.14.1 Detailed Description

Configuration class for the control loop simulation.

Stores and manages the simulation loop, signal generator, number of steps, and loop mode

5.15 Konfiguracja.h

[Go to the documentation of this file.](#)

```
00001
00006 #pragma once
00007 #include "PetlaUAR.h"
00008 #include "Sygnal.h"
00009 #include <memory>
00010 #include <string>
00011
00015 class Konfiguracja {
00016 private:
00017     std::unique_ptr<PetlaUAR> petla;
00018     std::unique_ptr<Sygnal> generator;
00019     int liczbaKrokow;
00020     bool zamkniete;
00021
00022 public:
00023
00027     Konfiguracja();
00028
00033     void wczytaj(const std::string& nazwaPliku);
00034
00039     void zapisz(const std::string& nazwaPliku) const;
00040
00041
00046     void ustawPetle(std::unique_ptr<PetlaUAR> nowaPetla);
00047     void ustawGenerator(std::unique_ptr<Sygnal> nowyGenerator);
00048     void ustawLiczbeKrokow(int kroki);
00049     void ustawZamkniete(bool stan);
00051
00056     PetlaUAR* pobierzPetle() const;
00057     Sygnal* pobierzGenerator() const;
00058     int pobierzLiczbeKrokow() const;
00059     bool czyZamkniete() const;
00061 };
```

5.16 ModelARX.h

```

00001 #pragma once
00002 #include "ObiektSISO.h"
00003 #include <vector>
00004 #include <deque>
00005 #include <random>
00006
00016 class ModelARX :
00017     public ObiektSISO
00018 {
00019 public:
00027     ModelARX(std::vector<double> a, std::vector<double> b, unsigned k, double zmienna);
00028
00034     ModelARX(const std::string& nazwaPliku);
00035
00036     double symuluj(double u) override;
00037
00043     void serializuj(const std::string& nazwaPliku) const;
00044
00045 private:
00046     std::vector<double> a;
00047     std::vector<double> b;
00048
00049     std::deque<double> u;
00050     std::deque<double> y;
00051
00052     unsigned k;
00053
00054     // Noise-related members
00055     double mocZaklocenia;
00056     std::default_random_engine generator;
00057     std::normal_distribution<double> rozklad{ 0.0, 1.0 };
00058
00059     // Serialization helpers
00060     std::vector<double> wspolczynnikiA;
00061     std::vector<double> wspolczynnikiB;
00062
00067     double generujZaklocenie() {
00068         return rozklad(generator) * mocZaklocenia;
00069     }
00070
00077     std::vector<double> parsujListe(const std::string& linia);
00078
00084     double zaklocenieTymczasowe()
00085     {
00086         return 0.0;
00087     }
00088 };

```

5.17 Nasycenie.h

```

00001 #pragma once
00002 #include "Sygnal.h"
00003
00004 #include <algorithm>
00005
00006 class Nasycenie : public Sygnal {
00007     std::unique_ptr<Sygnal> component;
00008     double max_amplitude;
00009 public:
00010     Nasycenie(std::unique_ptr<Sygnal> s, double max);
00011     double generuj(int t) override;
00012     void serializuj(std::ostream& out) const override;
00013 };

```

5.18 Programowanie_Objektowe_04/ObiektSISO.h File Reference

SISO (Single Input Single Output) system interface.

This graph shows which files directly or indirectly include this file:

Classes

- class [ObiektSISO](#)
Abstract SISO system.

5.18.1 Detailed Description

SISO (Single Input Single Output) system interface.

5.19 ObiektSISO.h

[Go to the documentation of this file.](#)

```
00001
00005 #pragma once
00006
00010 class ObiektSISO {
00011 public: // Change access specifier to public
00017     virtual double symuluj(double u) = 0;
00018
00019     virtual ~ObiektSISO() = default;
00020 };
```

5.20 Programowanie_Obiektowe_04/ObiektStatyczny.h File Reference

Static object model for control system simulation.

```
#include "ObiektSISO.h"
#include <string>
Include dependency graph for ObiektStatyczny.h:
```

Classes

- class [ObiektStatyczny](#)
Represents a static SISO object with output range limits.

5.20.1 Detailed Description

Static object model for control system simulation.

5.21 ObiektStatyczny.h

[Go to the documentation of this file.](#)

```
00001
00005 #pragma once
00006 #include "ObiektSISO.h"
00007 #include <string>
00008
00012 class ObiektStatyczny :
00013     public ObiektSISO
00014 {
00015     double dolny, gorny;
00016 public:
00017
00023     ObiektStatyczny(double d, double g);
00024
00029     ObiektStatyczny(const std::string& plik);
00030
00031     double symuluj(double x) override;
00032
00037     void serializuj(const std::string& plik) const;
00038 };
```

5.22 Programowanie_Objektowe_04/PetlaUAR.h File Reference

Control loop for automatic regulation system (UAR)

```
#include "ObiektSISO.h"
#include <vector>
#include <memory>
#include <string>
```

Include dependency graph for PetlaUAR.h: This graph shows which files directly or indirectly include this file:

Classes

- class [PetlaUAR](#)
Represents a control loop composed of multiple SISO components.

5.22.1 Detailed Description

Control loop for automatic regulation system (UAR)

5.23 PetlaUAR.h

[Go to the documentation of this file.](#)

```
00001
00005 #pragma once
00006 #include "ObiektSISO.h"
00007 #include <vector>
00008 #include <memory>
00009 #include <string>
00010
00015 class PetlaUAR :
00016     public ObiektSISO
00017 {
00018     std::vector<std::unique_ptr<ObiektSISO>> obiekty;
00019     std::vector<std::string> typy;
00020     bool zamknieta = true;
00021     double poprzednieWyjscie = 0.0;
00022
00023 public:
00024
00030     double symuluj(double x) override;
00031
00036     void ustawZamknieta(bool stan);
00037
00043     void dodajKomponent(std::unique_ptr<ObiektSISO> obiekt, const std::string& typ);
00044
00051     void dodajKomponentNaPozycji(std::unique_ptr<ObiektSISO> obiekt, size_t index, const std::string&
00052 typ);
00057     void usunKomponent(size_t index);
00058
00063     void serializuj(const std::string& nazwaPliku) const;
00064
00069     void deserializuj(const std::string& nazwaPliku);
00070
00075     std::vector<std::string> pobierzTypyKomponentow() const;
00076 };
```


5.24 Prostokat.h

```

00001 #pragma once
00002 #include "Sygnal.h"
00003
00004 class Prostokat : public Sygnal {
00005     double amplituda;
00006     int okres;
00007     double wypelnienie;
00008 public:
00009     Prostokat(double a, int o, double w);
00010     double generuj(int t) override;
00011     void serializuj(std::ostream& out) const override;
00012 };

```

5.25 Programowanie_Objektowe_04/RegulatorPID.h File Reference

PID controller implementation.

```

#include "ObiektSISO.h"
#include <stdexcept>

```

Include dependency graph for RegulatorPID.h:

Classes

- class [RegulatorPID](#)
PID controller with configurable modes (P/PI/PID)

5.25.1 Detailed Description

PID controller implementation.

5.26 RegulatorPID.h

[Go to the documentation of this file.](#)

```

00001
00005 #pragma once
00006 #include "ObiektSISO.h"
00007 #include <stdexcept>
00008
00012 class RegulatorPID :
00013     public ObiektSISO
00014 {
00015 public:
00020     explicit RegulatorPID(double k);
00021
00027     RegulatorPID(double k, double Ti);
00028
00035     RegulatorPID(double k, double Ti, double Td);
00036
00037     double symuluj(double uchyb) override;
00038
00039     RegulatorPID(const std::string& plik);
00040     void serializuj(const std::string& plik) const;
00041
00042 private:
00043     double k;
00044     double Ti;
00045     double Td;
00046     double suma_uchybow;
00047     double uchyb_poprzedni;
00048
00049     void sprawdzNastawy() const;
00050 };

```

5.27 Sinus.h

```

00001 #pragma once
00002 #include "Sygnal.h"
00003 #include <cmath>
00004
00005 class Sinus : public Sygnal {
00006     double amplituda, czestotliwosc;
00007     double y_prev, y_prev_prev;
00008     int last_t;
00009 public:
00010     Sinus(double a, double f);
00011     double generuj(int t) override;
00012     void serializuj(std::ostream& out) const override;
00013 };

```

5.28 Stala.h

```

00001 #pragma once
00002 #include "Sygnal.h"
00003
00004 class Stala : public Sygnal {
00005     double wartosc;
00006 public:
00007     Stala(double w);
00008     double generuj(int t) override;
00009     void serializuj(std::ostream& out) const override;
00010 };

```

5.29 Programowanie_Objektowe_04/Sygnal.h File Reference

Abstract base class for all signal types.

```

#include <iostream>
#include <memory>
#include <random>

```

Include dependency graph for Sygnal.h: This graph shows which files directly or indirectly include this file:

Classes

- class [Sygnal](#)
Interface for signal generators.

Functions

- `std::unique_ptr< Sygnal > deserializujSygnal (std::istream &in)`
Deserialize a signal object from input stream.

5.29.1 Detailed Description

Abstract base class for all signal types.

5.29.2 Function Documentation

5.29.2.1 deserializujSygnal()

```

std::unique_ptr< Sygnal > deserializujSygnal (
    std::istream & in)

```

Deserialize a signal object from input stream.

Parameters

<code>in</code>	Input stream
-----------------	--------------

Returns

`std::unique_ptr<Sygnal>` Deserialized signal

5.30 Sygnal.h

[Go to the documentation of this file.](#)

```

00001
00005 #pragma once
00006 #include <iostream>
00007 #include <memory>
00008 #include <random>
00009
00013 class Sygnal {
00014 public:
00015     virtual ~Sygnal() = default;
00016
00022     virtual double symuluj(int t) = 0;
00023
00028     virtual void serializuj(std::ostream& out) const = 0;
00029
00034     virtual std::unique_ptr<Sygnal> klonuj() const = 0;
00035 };
00036
00042 std::unique_ptr<Sygnal> deserializujSygnal(std::istream& in);

```

5.31 Programowanie_Obiektowe_04/SygnalStaly.h File Reference

Constant (DC) signal implementation.

```
#include "Sygnal.h"
```

Include dependency graph for SygnalStaly.h:

Classes

- class [SygnalStaly](#)
Constant value signal.

5.31.1 Detailed Description

Constant (DC) signal implementation.

5.32 SygnalStaly.h

[Go to the documentation of this file.](#)

```
00001
00005 #pragma once
00006 #include "Sygnal.h"
00007
00011 class SygnalStaly :
00012     public Sygnal
00013 {
00014     double wartosc;
00015 public:
00020     explicit SygnalStaly(double w = 0.0);
00021
00022     double symuluj(int t) override;
00023     void serializuj(std::ostream& out) const override;
00024     std::unique_ptr<Sygnal> klonuj() const override;
00025 };
```

5.33 SygnalZlozony.h

```
00001 #pragma once
00002 #include "Sygnal.h"
00003 #include <vector>
00004
00005 class SygnalZlozony : public Sygnal {
00006     std::vector<std::unique_ptr<Sygnal>> sygnaly;
00007 public:
00008     SygnalZlozony(std::vector<std::unique_ptr<Sygnal>> s);
00009     double generuj(int t) override;
00010     void serializuj(std::ostream& out) const override;
00011 };
```

5.34 SzumBialy.h

```
00001 #pragma once
00002 #include "Sygnal.h"
00003 #include <random>
00004
00005 class SzumBialy : public Sygnal {
00006     mutable std::default_random_engine generator;
00007     std::normal_distribution<double> rozklad;
00008 public:
00009     SzumBialy(double srednia, double odchylenie);
00010     double generuj(int t) override;
00011     void serializuj(std::ostream& out) const override;
00012 };
```

5.35 Programowanie_Objektowe_04/TestyKonfiguracji.h File Reference

Declaration of test functions for the control system components.

Functions

- void **testZapisulOdczytu** ()
Tests serialization and deserialization of configuration.
- void **testZmianyParametrow** ()
Tests parameter modification in the configuration.
- void **testPelnejSymulacji** ()
Runs a complete simulation to verify functionality.
- void **uruchomTesty** ()
Executes all test cases.

5.35.1 Detailed Description

Declaration of test functions for the control system components.

5.36 TestyKonfiguracji.h

[Go to the documentation of this file.](#)

```
00001
00005 #pragma once
00006
00010 void testZapisuIOdczytu();
00014 void testZmianyParametrow();
00015
00019 void testPelnejSymulacji();
00020
00024 void uruchomTesty();
```

5.37 Trojkat.h

```
00001 #pragma once
00002 #include "Sygnal.h"
00003
00004 class Trojkat : public Sygnal {
00005     double amplituda;
00006     int okres;
00007     double wypelnienie; // stosunek czasu narastania do całego okresu (0-1)
00008 public:
00009     Trojkat(double a, int o, double w);
00010     double generuj(int t) override;
00011     void serializuj(std::ostream& out) const override;
00012 };
```

